

Ejercicio Demostrativo

Sergio Sebastian Sandoval Ruiz

Tiempo estimado: 2 hrs/min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Estructura de Datos - C] hola

1. Marco Formativo

1.1 Valores

Resiliencia	Durante el ejercicio demostrativo, el estudiante puede cometer errores al implementar la pila o la cola. La resiliencia se evidencia al no rendirse ante un error de compilación o lógica, depurar el código con calma y volver a intentarlo hasta obtener el resultado correcto
--------------------	--

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Implementa estructuras de datos lineales y no lineales usando programación orientada a objetos para desarrollar soluciones eficientes.
Competencia Específica	Construye manualmente las estructuras Pila y Cola mediante nodos enlazados, aplicando correctamente las operaciones Push/Pop y Enqueue/Dequeue, y calcula la posición de elementos en una matriz utilizando mapeo lexicográfico Row-Major y Column-Major.

2. Palabras Clave

Pila (Stack): Estructura de datos lineal que sigue la política LIFO (Last In, First Out). Solo permite insertar y eliminar elementos por su extremo superior, llamado tope o cima.

Cola (Queue): Estructura de datos lineal que sigue la política FIFO (First In, First Out). Los elementos se insertan por el final y se extraen por el frente.

LIFO: Last In, First Out. El último elemento en ingresar es el primero en salir. Característica fundamental de la Pila.

FIFO: First In, First Out. El primer elemento en ingresar es el primero en salir. Característica fundamental de la Cola.

Push / Pop: Operaciones básicas de la Pila. Push agrega un elemento al tope; Pop elimina y retorna el elemento del tope.

Enqueue / Dequeue: Operaciones básicas de la Cola. Enqueue inserta al final; Dequeue elimina y retorna el elemento del frente.

Mapeo Lexicográfico: Técnica para calcular la dirección de memoria de un elemento en una matriz bidimensional usando fórmulas matemáticas según el orden de almacenamiento.

Row-Major: Orden de almacenamiento en matrices donde los elementos se guardan fila por fila. Fórmula: $Loc(i,j) = i \times nCols + j$.

Column-Major: Orden de almacenamiento en matrices donde los elementos se guardan columna por columna. Fórmula: $Loc(i,j) = j \times nRows + i$.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada la implementación manual de dos estructuras de datos fundamentales con políticas de acceso especial: la Pila (Stack) y la Cola (Queue), así como el cálculo de posiciones en una matriz mediante Mapeo Lexicográfico. El ejercicio corresponde a los contenidos de la Semana 5 del curso Estructuras de Datos [B].

La clase se orienta a que el estudiante comprenda la diferencia entre la política LIFO de la Pila y la política FIFO de la Cola, identifique cuándo aplicar cada estructura en un problema real, implemente correctamente los nodos y las operaciones desde cero sin usar librerías, y aplique las fórmulas de mapeo lexicográfico Row-Major y Column-Major para localizar elementos en una matriz.

4.Contenido

Descripción del ejercicio

El ejercicio demostrativo consiste en implementar desde cero, sin usar librerías estándar, dos estructuras de datos con políticas de acceso especial: una Pila (Stack) y una Cola (Queue), ambas mediante nodos enlazados. Adicionalmente, se resuelven ejercicios de Mapeo Lexicográfico aplicando las fórmulas Row-Major y Column-Major para localizar elementos dentro de una matriz bidimensional almacenada en memoria lineal.

El auxiliar desarrolla en vivo el código paso a paso mientras explica cada decisión de diseño, permitiendo al estudiante observar la lógica antes de replicarla en su propia práctica.

Objetivo de la demostración

Demostrar el procedimiento para implementar manualmente las estructuras Pila y Cola mediante nodos enlazados en Java, aplicando las operaciones Push/Pop y Enqueue/Dequeue con sus respectivas validaciones, con el fin de que el estudiante comprenda la diferencia entre la política LIFO y FIFO, identifique el rol de los punteros tope y frente/final, y sea capaz de replicar la implementación en su práctica individual.

Recursos necesarios

Software y entorno:

- Java JDK 11 o superior (OpenJDK o Oracle JDK)
- IDE: IntelliJ IDEA, Eclipse o VS Code con extensión Java
- Terminal / Consola para compilación y ejecución

Herramientas técnicas:

- Compilador javac incluido en el JDK
- Pizarrón o pantalla compartida para diagramar la estructura de nodos

Materiales de apoyo:

- Diapositivas de la Semana 5 con definiciones de Pila, Cola y Mapeo Lexicográfico
- Archivo base Stack.java y Queue.java con estructura de nodos ya definida (opcional)
- Tabla de ejercicios de mapeo lexicográfico impresa o proyectada

Configuración mínima:

- Java 11+, 4 GB RAM, proyector o monitor de demostración

Desarrollo paso a paso

Nota: El procedimiento está redactado para que cualquier tutor pueda reproducirlo en semestres futuros. Cada paso incluye qué hacer, por qué hacerlo y qué observar.

Paso 1 — Introducción conceptual (10 min)

Presentar las diapositivas de la Semana 5 con las definiciones de Pila y Cola. Dibujar en pizarrón la diferencia visual entre LIFO y FIFO: una pila de libros vs. una fila de personas. El estudiante debe comprender que la diferencia NO está en la estructura del nodo sino en la política de acceso.

Pregunta al estudiante: ¿En qué escenario de la vida real preferirías una pila? ¿y una cola? (Respuesta esperada: pila → historial de navegador / deshacer; cola → cola de impresión / atención al cliente).

Paso 2 — Implementación de la Pila (25 min)

Crear el archivo Stack.java. Comenzar con la clase NodoStack. Explicar que cada nodo tiene un dato y un puntero al siguiente nodo debajo de él.

```
class NodoStack {
    int dato;
    NodoStack siguiente;
}

class Stack {
    NodoStack tope = null;

    void push(int valor) {
        NodoStack nuevo = new NodoStack();
        nuevo.dato = valor;
        nuevo.siguiente = tope; // apunta al antiguo tope
        tope = nuevo;          // el nuevo nodo es ahora el tope
    }

    int pop() {
        if (tope == null) return -1; // pila vacía
        int valor = tope.dato;
        tope = tope.siguiente;       // avanzar al siguiente
        return valor;
    }

    int peek() { return tope == null ? -1 : tope.dato; }
    boolean isEmpty() { return tope == null; }
}
```

Demostrar en consola: push(10), push(20), push(30). Mostrar que pop() retorna 30, luego 20, luego 10. Dibujar en pizarrón el estado del tope tras cada operación.

Paso 3 — Implementación de la Cola (25 min)

Crear Queue.java. Explicar por qué se necesitan DOS punteros: frente (para dequeue) y finalCola (para enqueue). Sin el puntero al final, insertar al final sería $O(n)$ en vez de $O(1)$.

```
class NodoCola {
    int dato;
    NodoCola siguiente;
}

class Cola {
    NodoCola frente = null;
    NodoCola finalCola = null;

    void enqueue(int valor) {
        NodoCola nuevo = new NodoCola();
        nuevo.dato = valor;
        nuevo.siguiente = null;
        if (finalCola != null) finalCola.siguiente = nuevo;
        else frente = nuevo;      // cola vacía: frente = nuevo
        finalCola = nuevo;
    }

    int dequeue() {
        if (frente == null) return -1;
        int valor = frente.dato;
        frente = frente.siguiente;
        if (frente == null) finalCola = null; // cola quedó vacía
        return valor;
    }

    int peek() { return frente == null ? -1 : frente.dato; }
    boolean isEmpty() { return frente == null; }
}
```

Demostrar: enqueue(A), enqueue(B), enqueue(C). Mostrar que dequeue() retorna A, luego B, luego C. Enfatizar el caso borde: cuando la cola tiene un solo elemento, al hacer dequeue tanto frente como finalCola deben quedar en null.

Paso 4 — Mapeo Lexicográfico (20 min)

Presentar la tabla de fórmulas. Plantear una matriz de 3 filas × 4 columnas y calcular manualmente la posición del elemento en la fila 1, columna 2 usando ambas fórmulas.

Tipo	Fórmula	Significado	Ejemplo i=1, j=2, nCols=4, nRows=3
Row-Major	$\text{Loc}(i,j) = i \times \text{nCols} + j$	Elemento en fila i, columna j almacenado fila por fila	$1 \times 4 + 2 = 6$
Column-Major	$\text{Loc}(i,j) = j \times \text{nRows} + i$	Elemento en fila i, columna j almacenado columna por columna	$2 \times 3 + 1 = 7$

Ejercicio en vivo: pedir a un estudiante que calcule $\text{Loc}(0,3)$ en Row-Major con $\text{nCols}=4$. Respuesta: $0 \times 4 + 3 = 3$. Luego pedir $\text{Loc}(2,1)$ en Column-Major con $\text{nRows}=3$. Respuesta: $1 \times 3 + 2 = 5$.

Observaciones Importantes

- Error común en push: asignar `tope = nuevo` ANTES de `nuevo.siguiente = tope`. Esto pierde la referencia a la lista anterior. Siempre primero apuntar al antiguo tope, luego actualizar tope.
- En la Cola, olvidar actualizar `final_cola = null` cuando queda vacía tras un dequeue provoca que el siguiente enqueue conecte nodos en un estado inconsistente.
- En mapeo lexicográfico, los índices empiezan en 0. $\text{Loc}(0,0) = 0$ siempre en ambos esquemas.
- La diferencia entre Row-Major y Column-Major importa en rendimiento: accesos secuenciales en Row-Major son más rápidos en C/Java (caché friendly) al igual que Column-Major en FORTRAN/MATLAB.

4.5 Tabla Comparativa — Pila vs Cola

Operación	Pila (LIFO)	Operación	Cola (FIFO)	Complejidad
PUSH	Agrega elemento al tope de la pila	ENQUEUE	Inserta elemento al final de la cola	$O(1)$
POP	Elimina y retorna el elemento del tope	DEQUEUE	Elimina y retorna el elemento del frente	$O(1)$
PEEK	Consulta el tope sin eliminar	PEEK	Consulta el frente sin eliminar	$O(1)$
EMPTY	Verifica si la pila está vacía	EMPTY	Verifica si la cola está vacía	$O(1)$

SIZE	Retorna el número de elementos	SIZE	Retorna el número de elementos	O(1)
-------------	--------------------------------	-------------	--------------------------------	------

4.6 Resultado Esperado

Al finalizar el ejercicio demostrativo, el estudiante debe ser capaz de observar el comportamiento correcto de ambas estructuras en consola, verificar que la política LIFO y FIFO se cumple con los datos de prueba, y comprender por qué se necesitan los punteros tope (Pila) y frente/finalCola (Cola).

Elemento	Resultado Esperado
Pila — push(10), push(20), push(30)	Estado: tope → [30] → [20] → [10]. pop() retorna 30, luego 20, luego 10.
Cola — enqueue(A), enqueue(B), enqueue(C)	Estado: frente → [A] → [B] → [C]. dequeue() retorna A, luego B, luego C.
Mapeo Row-Major i=2, j=3, nCols=5	Loc(2,3) = $2 \times 5 + 3 = 13$ — posición 13 en el arreglo lineal.
Mapeo Column-Major i=2, j=3, nRows=4	Loc(2,3) = $3 \times 4 + 2 = 14$ — posición 14 en el arreglo lineal.

4.7 Aplicación Práctica

Las Pilas se utilizan en el manejo del historial de navegación web (botón Atrás), en la ejecución de funciones recursivas (call stack del sistema operativo), en algoritmos de balanceo de paréntesis y en la funcionalidad Deshacer/Rehacer de editores de texto.

Las Colas se aplican en sistemas de impresión compartida, colas de atención al cliente, procesamiento de solicitudes en servidores web, y simulación de eventos discretos. En el proyecto del curso (EDD MedTrack / Cinema), la Cola de prioridad gestiona las solicitudes especiales de clientes.

El Mapeo Lexicográfico es la base para entender cómo los lenguajes de programación almacenan matrices en memoria, lo cual impacta directamente en el rendimiento de algoritmos de procesamiento de imágenes, matrices de transformación y cómputo científico.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- El orden de las instrucciones en `push()` (primero `nuevo.siguiente = tope`, luego `tope = nuevo`) es el punto donde más estudiantes se confunden. Vale la pena dibujar el estado de la memoria antes y después de cada línea.
- En la Cola, el caso borde donde al hacer `dequeue` la cola queda vacía (`frente = null` → también `finalCola = null`) suele olvidarse y genera `NullPointerException` en el siguiente `enqueue`.
- Las fórmulas de mapeo lexicográfico se memorizan fácil, pero los estudiantes suelen confundir `nRows` con `nCols` en cada fórmula. Insistir en que Row-Major usa `nCols` (columnas de la fila) y Column-Major usa `nRows` (filas de la columna)

Recomendaciones para futuras sesiones:

- Iniciar dibujando en pizarrón la estructura de nodos antes de abrir el IDE. Los estudiantes entienden mejor el código si ya tienen la imagen mental del nodo con sus punteros.
- Preparar un archivo `Main.java` con las llamadas de prueba ya escritas para no perder tiempo durante la demostración. Solo el código de la estructura se escribe en vivo.
- Dedicar los últimos 10 minutos a un ejercicio rápido de mapeo lexicográfico donde los estudiantes calculen en papel antes de que el auxiliar muestre la respuesta.
- Tener a la mano la diapositiva con la tabla comparativa Pila vs Cola para responder preguntas sobre cuándo usar cada estructura.

Puntos de mayor dificultad para el estudiante:

- Entender por qué la Pila necesita solo un puntero (`tope`) y la Cola necesita dos (`frente` y `finalCola`).
- El caso borde de `dequeue` en una cola de un solo elemento.
- No confundir Row-Major con Column-Major al aplicar las fórmulas.

6. Referencias

1. GeeksforGeeks. (2023). Stack Data Structure. Recuperado de <https://www.geeksforgeeks.org/stack-data-structure/>
2. GeeksforGeeks. (2023). Queue Data Structure. Recuperado de <https://www.geeksforgeeks.org/queue-data-structure/>
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. Capítulo 10: Estructuras de datos elementales.

- 
4. Weiss, M. A. (2013). Data Structures and Algorithm Analysis in Java (3rd ed.). Pearson. Capítulo 3: Listas, Pilas y Colas.
 5. Material de cátedra: Diapositivas Semana 5, EDD [B] — Estructuras con Políticas de Acceso Especial. Escuela de Ingeniería en Ciencias y Sistemas, USAC, 2026.